1. Ein Speicherort für alle: Was bedeutet `static` ?

In C# besitzt normalerweise jedes Objekt seine eigenen Daten im Speicher. Wenn mehrere Objekte aus derselben Klasse erstellt werden, hat jedes Objekt seine eigene Kopie der Eigenschaften.

```
class Obstacle
{
    public string Design;
}
```

Wenn Design **nicht static** ist, dann gilt:

- Jedes Hindernis besitzt seinen eigenen Speicherort.
- Jedes Hindernis kann anders aussehen.

```
Obstacle o1 = new Obstacle();
o1.Design = "#";

Obstacle o2 = new Obstacle();
o2.Design = "*";
```

Statische Eigenschaften und Methoden gehören nicht einzelnen Objekten, sondern der gesamten Klasse. Es gibt dann nur noch **einen gemeinsamen Speicherort**, auf den alle Objekte zugreifen.

```
class Obstacle
{
    public static string Design;
}
```

Jetzt gibt es nur noch einen gemeinsamen Speicherort für Design. Wenn du den Wert änderst:

```
Obstacle.Design = "#";
```

Dann ändern alle Hindernisse ihr aussehen.

static bedeutet: Ein Speicherort für alle Objekte der Klasse.

2. Warum gemeinsame Speicherorte? – Wozu gibt es static?

Gemeinsame Speicherorte werden genutzt, wenn bestimmte Informationen für alle Objekte gleich sein sollen.
Typische Anwendungsfälle sind:

- Highscores
- globale Zustände

Statt diese Werte mehrfach zu speichern, werden sie zentral abgelegt. Das spart Speicher, vermeidet Fehler und erleichtert die Verwaltung.

Beispiel

Im RunningMan-Spiel kann ein gemeinsames Hindernis-Design oder eine globale Spielgeschwindigkeit sinnvoll sein.

3. Gemeinsame Speicherplätze: static bei Variablen

Eine statische Eigenschaft existiert im gesamten Programm nur einmal. Alle Objekte der Klasse greifen auf denselben Wert zu.

Nicht static:

- Jedes Hindernis kann anders aussehen.

Static:

- Alle Hindernisse haben dasselbe Aussehen.
- Der Zugriff erfolgt immer über den Klassennamen:

Name der Klasse

```
Obstacle.Design = "#";
```

4. Aktionen ohne Spielfigur: static bei Methoden

Statische Methoden gehören ebenfalls zur Klasse und nicht zu einzelnen Objekten. Sie können ohne Objekterstellung aufgerufen werden.

```
class Obstacle
{
    public static string Design;

    public static void ChangeGlobalDesign(string newDesign)
    {
        Design = newDesign;
    }
}
```

5. Die Schaltzentrale: Static-Klassen

Eine static-Klasse ist eine Klasse, von der keine Objekte erzeugt werden können

(Der new Operator funktioniert nicht).

Sie dient als Zentrale für gemeinsame Daten und Funktionen.
Wichtige Regeln:

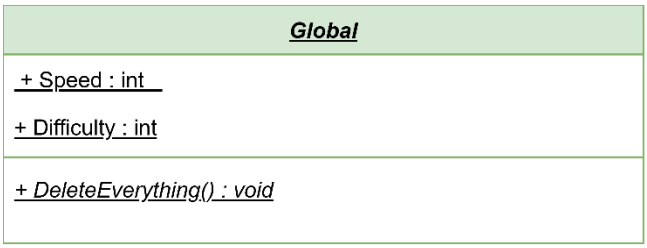
- Von static-Klassen kann kein Objekt erstellt werden
- Alle Member müssen static sein
- Sie enthalten nur Klassenlogik, keine Objektdaten

```
static class Global
{
    public static int Speed;
    public static int Difficulty;
}
```

Diese Klasse verwaltet zentrale Spielwerte.

6. Der Bauplan des Spiels: static im UML-Diagramm

In UML-Klassendiagrammen werden statische Eigenschaften und Methoden unterstrichen dargestellt.



7. Die RunningMan-Zentrale: static im Projekt

Zwei typische Möglichkeiten für statische Klassen sind eine Klasse

- `static class Globa`
- `static class Score`

Implementiere diese Klassen in dein Projekt RunningMan ein.

8. D Prüfe dein Wissen

a) Was unterscheidet eine statische Klasse von einer nicht-statischen Klasse?
b) Wie ruft man die Member einer statischen Klasse auf?
c) Was gilt für alle Member einer statischen Klasse?
d) Wann macht es Sinn, eine Klasse statisch zu machen?
e) Kann es eine nicht statische Klasse, statische Member besitzen?
f) Kann eine statische Klasse, nicht statische Member besitzen?
g) Können in einer nicht-statischen Klasse auch statische Member enthalten sein?

9. Bonus-Level (nur für Super-Profis): Das Singleton-Pattern

static stellt gemeinsame Speicherorte bereit, ohne Objekte zu erzeugen.
Manchmal möchte man jedoch genau eine Instanz einer Klasse verwenden. In diesem Fall nutzt man das Singleton-Pattern.

```
public class GameManager
{
    public static GameManager Instance { get; } = new GameManager();

    private GameManager() { }
}
```

Unterschied:

- static → keine Instanz
- Singleton → genau eine Instanz

Beide ermöglichen zentrale Verwaltung, aber auf unterschiedliche Weise.